

API Testing with SoapUI

Тестування SOAP API — CountryInfoService, Custom Properties, асерти, баж-репорт та порівняння SoapUI vs Postman

Автор: Анна Гуртова (Anna Hurtova)

Курс: Beetroot Academy / Helvetas Ukraine ("Стійкість")

Роль: Trainee QA Manual Engineer

Дата: Серпень 2026

Структура тестового набору (TestSuite)

Інструмент: SoapUI v5.8.0 | WSDL: [CountryInfoService.wsdl](#)

Мета: Покриття API-методів SOAP-сервісу CountryInfoService позитивними та негативними автоматизованими сценаріями з динамічною параметризацією через Custom Properties та автоматичними асертами.

На основі імпортованої WSDL-схеми створено **4 TestCase** для обов'язкових операцій, кожен з яких містить по **4 тестових степах** (2 позитивних + 2 негативних) = **16 степів, 48+ асертів**:

TESTCASE	ОПЕРАЦІЯ	ПОЗИТИВНІ	НЕГАТИВНІ	ВСЬОГО СТЕПІВ
1	CapitalCity	2 (UA → Kyiv, US → Washington)	2	4
2	CountryISOCCode	2	2 (BUG-001 знайдено)	4
3	CountryCurrency	2	2	4
4	CountriesUsingCurrency	2	2	4

Параметризація даних (Custom Properties)

Замість статичних даних у XML-запитах впроваджено динамічну параметризацію. На рівні кожного TestCase створено набір змінних (Custom Properties), для кожного ступу задіяно мінімум 2 змінні:

- Вхідне значення** для XML-тегу запиту (наприклад, `sCountryISOCCode`)
- Очікуваний результат** для валідації в асертах (наприклад, `ExpectedResult`)

Значення пов'язані з тегами XML через контекстний синтаксис SoapUI:

```
<web:sCountryISOCCode>${#TestCase#CountryCode1}</web:sCountryISOCCode>
```

Покриття перевірками (Assertions)

До кожного з 16 тестових степів застосовано мінімум **3 обов'язкових асерти**:

- Valid HTTP Status Codes** — очікуваний код стану 200 OK (успішність доставки HTTP-пакета)
- Not SOAP Fault** — валідація відсутності системних збоїв на рівні SOAP-протоколу
- Contains** — динамічна перевірка вмісту відповіді: порівняння фактичної відповіді з очікуваним значенням `${#TestCase#ExpectedResult}`

Тестові сценарії

Позитивні сценарії

По 2 кроки на кожну операцію з валідними даними:

CapitalCity Positive

- 1 `${#TestCase#CountryCode1}` = UA → Expected: Kyiv 2 `${#TestCase#CountryCode2}` = US → Expected: Washington

CountryISOCode Positive

- 1 CountryName → Expected ISO Code 2 CountryName → Expected ISO Code

CountryCurrency Positive

- 1 CountryCode → Expected Currency 2 CountryCode → Expected Currency

CountriesUsingCurrency Positive

- 1 CurrencyCode → Expected Countries 2 CurrencyCode → Expected Countries

Негативні сценарії

По 2 кроки з некоректними вхідними даними (неіснуючі коди, цифрові значення, порожні поля) для перевірки відмовистості бекенду:

CapitalCity Negative

- 3 Невалідний код → Expected: «Country not found» 4 Порожнє значення → Expected: «Country not found»

CountryISOCode Negative ⚠ BUG-001

- 3 Невалідне ім'я → Expected: «No country found» 4 Порожнє значення → **BUG: повертає «AX» замість помилки**

CountryCurrency Negative

- 3 Невалідний код → Expected: помилка 4 Порожнє значення → Expected: помилка

CountriesUsingCurrency Negative

- 3 Невалідний код → Expected: помилка 4 Порожнє значення → Expected: помилка

Звіт про виявлений дефект

BUG-001 Некоректне повернення ISO-коду «AX» при порожньому параметрі назви країни **Minor**

Operation: CountryISOCode

Preconditions: SoapUI v5.8.0 запущено; WSDL-проект імпортовано; Custom Property CountryName4 налаштовано з порожнім значенням.

Steps:

- Відкрити тестовий крок «CountryISOCode – Step 4 (Negative)».
- Перевірити, що в XML-тег передається порожня змінна:

```
<web:sCountryName>${#TestCase#CountryName4}</web:sCountryName>
```

- Надіслати SOAP-запит на сервер.

Actual Result: Бекенд виконує некоректну вибірку з БД та повертає ISO-код Аландських островів:

```
<m:CountryISOCodeResult>AX</m:CountryISOCodeResult>
```

Асерт Contains — **FAILED:** Missing token «No country found by that name» in Response.

Expected Result: При порожньому параметрі сервер НЕ повинен повертати ISO-код реальної країни. Очікується: «No country found by that name» або SOAP Fault.

Component: Backend Logic / Database Query Validation

Environment: macOS Sonoma, SoapUI v5.8.0 (ARM64)

Severity: Minor — порушено бізнес-логіку роботи з БД | **Priority:** Medium — потребує виправлення логіки обробки вхідних даних

Порівняльний аналіз: SoapUI та Postman

На основі практичного досвіду роботи з обома інструментами проведено їх порівняльний аналіз.

Робота з протоколами та форматами

Postman — орієнтований на REST API та JSON. Тестувати SOAP/XML можливо, але вимагає ручного формування XML-конверту та заголовків (Content-Type, SOAPAction).

SoapUI — створений для SOAP/WSDL сервісів. Автоматичне зчитування WSDL, генерація структури папок та XML-запитів із заготовленими тегами.

Assertions (Перевірки)

Postman — для будь-якого асерту необхідно писати JavaScript-код у вкладці Tests.

SoapUI — вбудована панель візуальних перевірок: Valid HTTP Status, Not SOAP Fault, Contains — кількома кліками без коду.

Змінні (Параметризація)

Postman — змінні через {{variable}}, зручний інтерфейс Environment/Collection.

SoapUI — Custom Properties з синтаксисом \${#TestCase#Variable}, складніший, чутливий до прихованих пробілів.

КРИТЕРІЙ	SOAPUI	POSTMAN
Плюси	Авто-генерація XML з WSDL; візуальні асерти без коду; ідеальний для Enterprise SOAP	Сучасний інтуїтивний UI; найкращий для REST/JSON; зручний експорт/імпорт колекцій
Мінуси		

КРИТЕРІЙ	SOAPUI	POSTMAN
	Застарілий Java UI; сумісність з Apple Silicon; чутливість до пробілів у Properties	Потреба писати JS для Tests; ручне формування XML-конверту для SOAP
Призначення	SOAP/WSDL вебсервіси, складні XML-інтеграції	REST API для сучасних веб-сайтів та мобільних додатків

Підсумок: Для сучасного QA-інженера важливо володіти обома інструментами. **Postman** — легкий універсальний інструмент для щоденної роботи з REST/JSON. **SoapUI** — потужний спеціалізований інструмент для складних SOAP/WSDL сервісів та XML-інтеграцій.